

Computational Physics and Data Science Exercise 2: Fresnel Diffraction

Oscar Purrington

School of Physics, University of Bristol.

(Dated: December 3, 2025)

The objective of this exercise was to investigate Fresnel and Fraunhofer diffraction patterns, and the relationship between the physical set-up and the results we see. We found that the pattern is governed by the Fresnel number F , proportional to ratio of the square aperture size to the observation distance, quantifying our physical parameters into predictable results. The main computational methods used were a direct numerical integration via the Gaussian Quadrature method, and a statistical approximation via the Monte Carlo integration. We find that the Gaussian Quadrature produces accurate results but scales with $O(N^2)$ as resolution increases, causing large runtimes, where as the Monte Carlo produces results with significantly quicker speed, but retains statistical noise leaving some artifacts in the produced images.

I. INTRODUCTION

The primary goal of this exercise is to simulate the diffraction patterns that are formed when light is passed through an aperture. Various aperture shapes will be used in 1-d or 2-d simulations. The governing equation for the observed behaviour is the Fresnel-Kirchhoff diffraction integral.

$$E(x, y, z) = \frac{e^{ikz}}{i\lambda z} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} E(x', y') \exp \left\{ \frac{ik}{2z} [(x - x')^2 + (y - y')^2] \right\} dx' dy' \quad (1)$$

This equation calculates the value of the electric field E at a given point on a screen at a distance z . This distance can be varied to see two major approximations of the diffraction pattern: Fresnel (or near-field), where the diffraction pattern changes with z , and Fraunhofer (or far-field), where the diffraction pattern is constant. The Fresnel-Kirchhoff integral is complex to solve analytically, giving us good reason to use this as an exercise in numerical integration. It also involves usage of complex numbers, something that cannot be handled directly by our main integrator `scipy.integrate.dblquad`. Therefore, splitting the integral into its real and imaginary parts (using the property of complex numbers that the cosine part is real and the sine part is imaginary), and recombining them to find the intensity values that are proportional to the square of the electric field strength must be employed. The main `scipy.integrate.dblquad` function uses Gaussian quadrature to generate highly precise numerical solutions to our integral for many shapes of aperture. An alternative approximation is used in part 4 in response to the main method being computationally intensive. This is a Monte Carlo approximation deriving the response from random sampling in a bounding box.

II. THEORY

Diffraction is the spreading out of waves as they pass through a gap or around an obstacle. The intensity of this spread is determined by the aperture size and the wavelength (if $\lambda \approx d$, then diffraction is maximised.) The primary equation that we can use to calculate the electric field given $E(x, y, z)$ on the plane of observation is the Fresnel-Kirchhoff integral [1]. This integral is a superposition of individually diffracted waves leaving every point of the aperture. Exact calculation of wave propagation involves a distance term, $r = \sqrt{Z^2 + (x - x')^2 + (y - y')^2}$ between each point on the aperture and each point on the observation plane. Calculating the phase shift kr using this is impossible to integrate over the aperture. We use an approximation known as the Fresnel Approximation.

$$r \approx Z + \frac{(x - x')^2 + (y - y')^2}{2Z} \quad (2)$$

This is derived from the first two terms of the binomial expansion. We then use this in the Fresnel Integral to give the $e^{ikz} \cdot \exp \left\{ \frac{ik}{2z} [(x - x')^2 + (y - y')^2] \right\}$. We take the first term out of the integral and use the property of the exponential form of the imaginary number that the cosine of the index is the real part, and the sine is the imaginary.

The Fresnel (Near-Field) Regime is the primary regime we are studying in this exercise. It occurs at small Z values, where the complex phase term is non-negligible. In this regime, varying Z varies the pattern we witness. Fraunhofer (Far-Field) regime behaviour occurs for large Z , simplifying the phase term and leading to a constant behaviour. This exercise also uses various computational methods. Gaussian Quadrature is a numerical integration method used by `scipy.integrate.dblquad` to perform a highly accurate and efficient integration for non-oscillating functions. Gaussian Quadrature involves approximating a definite integral by summing various points on the curve multiplied by a weighting assigned to the function value. An N -point integration can integrate a polynomial of degree up to $2N - 1$. `scipy.integrate.dblquad` uses an adaptive form. This works by splitting the integral into sub-integrals, estimating these and their error, and if the error is too large, further divides the sub-integrals into more sub-integrals until the final result is less than the required tolerance. A second method for numerical approximation is used in part 4, namely Monte Carlo integration. This approximates the integral by sampling random points within the area determined by the bounds, $\int f(x)dx \approx \text{Area} \cdot \frac{1}{N} \sum_{i=1}^N f(x_i)$. This allows for a good approximation over complex boundary shapes with a limitation of a slow error convergence rate ($O\left(\frac{1}{\sqrt{N}}\right)$) and a noisy result for low N .

III. CODE EXPLANATION

The code begins with a list of imports.

```
from scipy import integrate
import numpy as np
import matplotlib.pyplot as plt
```

`scipy.integrate` is the main library we use for this exercise, as it contains the `dblquad()` function, our adaptive Gaussian Quadrature integration method. `numpy` is used for handling mathematical functions such as the crucial sine and cosine functions, as well as providing square roots, π , and generating arrays of values for the pixels. Finally we import `matplotlib.pyplot` in order to produce graphs and images of our results, using `plt.plot()` and `plt.imshow()`. We then set various initial variables

```
# Variables #
LAMBDA = 6e-7 # Wave length [m]
K = 2 * np.pi / LAMBDA # Wave Number [1/m]
E0 = 60 # Electric Field Strength [V/m]
Z = 0.02 # Observation distance [m]
aperture_size = 2e-5 # Aperture Size (width) [m]
xp1, xp2 = ( # Sets Boundaries of Aperture
    -aperture_size / 2, #
    aperture_size / 2, #
) #
yp1, yp2 = xp1, xp2
FACTOR = (K * E0) / (2 * np.pi * Z) # Factor on Integral#
R = 1e-5 # Circular Aperture Radius [m]
```

We set a value for λ in the visible light spectrum (6×10^{-7} m here corresponding to the green section of the visible light spectrum.) We use this `lambda` value in the calculation of K , the wave number. This is used in the main integral. We then set an arbitrary value for the initial electric field strength, in order to have the scale of the graph in part 1 be more readable. Z is one of our primary variables. It represents the distance from the aperture to the observation plane. `aperture_size` is a variable that defines the width of the aperture opening, and allows us to easily set its bounds `xp1`, `xp2`, `yp1`, `yp2`. `FACTOR` is a value that appears outside of the integral, and is effectively just a multiplier that does the correct scaling. Finally, `R` defines the radius of our circular aperture that we use in parts 3 and 4. Two key functions we use repeatedly are the following.

```
def Fresnel2dreal(yp, xp, y, x, k, z):
    """Evaluates the Real part of the Integrand"""
    distance = (x - xp) ** 2 + (y - yp) ** 2
    phase = (k / (2 * z)) * distance

    return np.cos(phase)
```

```
def Fresnel2dimag(yp, xp, y, x, k, z):
    """Evaluates the Imaginary part of the Integrand"""
    distance = (x - xp) ** 2 + (y - yp) ** 2
    phase = (k / (2 * z)) * distance

    return np.sin(phase)
```

These functions use the property of the exponential function representation of imaginary numbers to easily calculate the real and imaginary parts of the Fresnel integral without directly dealing with imaginary numbers, something that `scipy.integrate.dblquad` cannot do.

The next two functions are used in part 3. The `scipy.integrate.dblquad` function calculates a double integral. The inner limits must be functions of the outer limits for a specific shape to be integrated over. These functions do this, calculating the upper and lower y' limits for a given x' to generate the circular aperture that we require. The `max(0, ...)` function is a stability measure. Occasionally due to floating point error the argument can be negative. If this occurs, the function call obviously fails. To prevent this, if the calculated limit is less than zero, zero itself is used instead.

```
def ypfunc(xp):
    """Evaluates the lower y' limit of an x'."""
    return -np.sqrt(max(0, R**2 - xp**2))

def yp2func(xp):
    """Evaluates the upper y' limit of an x'."""
    return np.sqrt(max(0, R**2 - xp**2))
```

Part 1 begins with a 2-d rectangular aperture. We initially create an array of x values spaced over the screen size using `np.linspace()`. Then we use a for loop looping over each of these values. The y value is set to zero, as this exercise is creating a 1-d diffraction pattern, so a constant y is required. We then parse `integrate.dblquad()` with our `Fresnel2dreal` and `Fresnel2dimag` functions, the horizontal bounds of the rectangle, a lambda function for the vertical bounds (this returns a constant upper and lower limit of integration regardless of the x_p value, to lock the shape to a rectangle. The lambda function is a requirement of `dblquad`, as it must take a function as input in these cases), and then the arguments to the Fresnel functions in the required order. These two calculations generate the real/imaginary parts of the integral, as well as their estimated error (`dblquad` must always generate the error, even if we don't use it). We then calculate the intensity using the property that $I \propto |E^2|$, and append it to the array. We then use a simple plotting method with `matplotlib.pyplot` to produce a graph of our result.

```
def part1():
    """Computes the diffraction pattern of light through a 2-d Aperture
    in 1 Dimension (as an intensity value), using a split-integration method
    using scipy.integrate.dblquad."""

    # Generates an array of numpoints x values
    numpoints = 100
    screen_size = 0.005
    # spaced over the screen size to contain the
    xvals = np.linspace(-screen_size, screen_size, numpoints)
    amplitudes = [] # area of interest.
    for x_pixel in xvals: # Loops each x value
        y_pixel = 0 # Sets y to zero (1-d pattern)
        #
        real, real_error = integrate.dblquad( # Parses integrate.dblquad to
            Fresnel2dreal, # obtain the real part of the
            # integral, and its error (dblquad
            xp1,
            xp2, # always returns this value)
            lambda xp: yp1, # The Lambda function returns
            lambda xp: yp2, # a constant upper and lower limit
```

```

    args=(y_pixel, x_pixel, K, Z),      # of integration, for any xp.
    )                                  #
    #
    imag, imag_error = integrate.dblquad( # Parses integrate.dblquad to
        Fresnel2dimag,                  # obtain the imaginary part
        xp1,                             # of the integral, in the same
        xp2,                             # manner as the real.
        lambda xp: yp1,                  #
        lambda xp: yp2,                  #
        args=(y_pixel, x_pixel, K, Z),   #
    )                                    #

    # Calculates each intensity value
    amplitude = (FACTOR * real) ** 2 + (FACTOR * imag) ** 2
    # Appends it to the array
    amplitudes.append(amplitude)

plt.figure()                          # Simple plotting function
plt.plot(xvals, amplitudes, label="Intensity") # to plot each intensity
plt.title(                             # against its corresponding
    "Fresnel Diffraction in One Dimension" # x value
    f"\n from a Square Aperture (z={Z}m)") #
plt.xlabel("Screen Position")           #
plt.ylabel("Relative Intensity")        #
plt.legend()                           #
plt.plot()
return

```

Part 2 is similar to Part 1, but instead generating a 2-d image. This code was given in the exercise, but will still be explained. The variables are set much as before to form a rectangular aperture, and 2 1-d arrays are created using `np.linspace` to store the x and y values of the screen coordinates. 2 2-d arrays are then created by copying the 1-d arrays into a grid to represent each coordinate on the screen using `np.meshgrid`. This is an example of vectorisation, that allows us to parse the sinc function onto each coordinate at the same time, rather than using nested for loops, a much more time-inefficient method. We compute the Fraunhofer limit using the cardinal sine function $\text{sinc} = \sin(x)/x$, as for large Z , the solution effectively becomes the fourier transform of the region, which for this case of a rectangular aperture, is the sinc function. We then use `plt.imshow` to generate a 2-d image of the pattern we simulate. This works by going through the array, seeing the value of intensity at each pixel, and applying a colour map depending on the value.

```

def part2():
    '''Computes the diffraction pattern of light through a rectangular 2-d aperture
    Note: Code was directly taken from exercise, with constants renamed for
    consistency. '''

    numpoints = 200
    Z = 0.1
    LAMBDA = 589e-9
    constant = np.pi*aperture_size/(LAMBDA*Z)
    x1 = -0.01
    x2 = x1
    y1 = x1
    y2 = x2

    # Create 1-D arrays for screen coordinates
    xvals = np.linspace(x1, x2, numpoints)
    yvals = np.linspace(x1, x2, numpoints)

    # Generate two 2-D arrays representing the x and y values on the screen
    X, Y = np.meshgrid(xvals, yvals)

    # Compute ideal 2-D diffraction from aperture using the sinc function (Fraunhofer limit)
    # Note: 'constant' must be defined earlier (likely constant = np.pi * aperture_width / (wavelength * screen_dis

```

```

intensity = (np.sinc(constant * X) * np.sinc(constant * Y))**2

# Define the limits for the plot
extents = (x1, x2, y1, y2)

# Plot the intensity array as an image
plt.imshow(
    intensity,
    vmin=0.0,
    # Note: The original code has a typo here (vmax=1.0-intensity.max())
    vmax=1.0 * intensity.max(),
    extent=extents,
    origin="lower",
    cmap="nipy_spectral_r" # Example color map
)

# Add labels and title
plt.xlabel("x (m)")
plt.ylabel("y (m)")
plt.title('Rectangular diffraction;\nz = {:.2f}m'.format(Z))
plt.colorbar()
plt.show()
return

```

The third part expands on Part 2 by this time using a circular aperture. Instead of directly creating a circular aperture, which is very difficult, we create a square of side length $2R$ which contains our circular aperture. x values and y values are created as before, but we have to use nested for loops to parse `integrate.dblquad` for each pixel, using our `ypfunc()` and `yp2func()` functions to define the upper and lower limits of integration. We then calculate and append the intensities as before, and display once again using `plt.imshow`. It is beneficial to include a progress counter as the runtime of this part can become rather long for a high `numpoints`, so a counter is displayed once every row of pixels is calculated.

```

def part3():
    """Evaluates 2-d diffraction through a circular aperture by creating functions
    for the y values to form a circular aperture."""
    ###
    # Creates an array of numpoints x and y values within the screen size,
    # and initialises a 2-d array for the intensity values.
    ###
    numpoints = 20
    screen_size = 0.005
    xvals = np.linspace(-screen_size, screen_size, numpoints)
    yvals = xvals
    amplitudes = np.zeros((numpoints, numpoints))

    ###
    # A nested for loop loops each value in the array, finding their intensity.
    # ypfunc() and yp2func() find the limits of integration for a given x,
    # allowing a circle boundary to be found.
    ###
    for j, y in enumerate(yvals):
        for i, x in enumerate(xvals):
            real, real_error = integrate.dblquad(
                Fresnel2dreal, -R, R, ypfunc, yp2func, args=(y, x, K, Z))
            imag, imag_error = integrate.dblquad(
                Fresnel2dimag, -R, R, ypfunc, yp2func, args=(y, x, K, Z))
            ###
            # Calculates the intensity and appends to the array
            ###
            amplitude = (FACTOR * real)**2 + (FACTOR * imag)**2
            amplitudes[j, i] = amplitude

```

```

    ###
    # Shows a progress bar, as runtimes can be quite long.
    ###
    print(f"progress: {j+1}/{numpoints}")

# Plotting function taken from given code in part2().
plt.figure()
extents = (-screen_size, screen_size, -screen_size, screen_size)
plt.imshow(
    amplitudes,
    vmin=0.0,
    vmax=1.0 * amplitudes.max(),
    extent=extents,
    origin="lower",
    cmap="nipy_spectral_r"
)
plt.xlabel("x (m)")
plt.ylabel("y (m)")
plt.title('Circular Diffraction;\nz = {:.2f}m'.format(Z))
plt.colorbar()
plt.show()
return

```

The final part effectively does the same thing as part 3, but using an approximative method. We set a number of random samples to average an intensity value for each point, and an area over which to sample. This area is a square of side $2R$ encompassing the circular region of interest. We then set as before the x and y values for the screen. As in part 3, a nested for loop is used to loop over each pixel, generating its random values. These distance from the origin of each pixel is calculated, and a simple circular mask applied, where if the distance is not within the defined circular bound, the intensity value is set to zero. The integral is calculated by then multiplying the area of the square by the outputs of the Fresnel 2-d functions, a vectorised process as it occurs over all the randomised points at once, and the amplitudes calculated as in every other part.

```

def part4():
    """Approximates a 2-d circular diffraction pattern using a Monte Carlo
    approximation for the integral. This is much quicker than a direct integration
    but retains a lot of noise for smaller values of N and numpoints"""
    N = 1000          # Number of random samples per points.
    Area = (2*R)**2   # Area of square encompassing circular region of interest
    numpoints = 100   # Number of pixels in the width of the square
    screen_size = 0.005 # Size of the screen

    xvals = np.linspace(-screen_size, screen_size,
                        numpoints) # Generates arrays for
    yvals = xvals                # x and y values, and
                                # for intensities.
    amplitudes = np.zeros((numpoints, numpoints))

    ###
    # Nested for loop loops each pixel, generating N random values for each.
    # Pixels with an r2 value <= to R**2 are counted, as they lie within our
    # disc. The real and imaginary parts are computed, and then the in_aperture
    # mask applied. Amplitude is calculated as before and plotting function
    # from part2() used.
    ###
    for j, y in enumerate(yvals):
        for i, x in enumerate(xvals):
            x_random = np.random.uniform(-R, R, N)
            y_random = np.random.uniform(-R, R, N)
            r2 = x_random**2+y_random**2
            in_aperture = r2 <= R**2

```

```

real = Fresnel2dreal(y_random, x_random, y, x, K, Z)
imag = Fresnel2dimag(y_random, x_random, y, x, K, Z)

real = real * in_aperture
imag = imag * in_aperture

real_integral = Area * np.mean(real)
imag_integral = Area * np.mean(imag)

amplitude = ((FACTOR * real_integral)**2 +
              (FACTOR * imag_integral)**2 )
amplitudes[j, i] = amplitude
print(f"progress: {j+1}/{numpoints}")

plt.figure()
extents = (-screen_size, screen_size, -screen_size, screen_size)
plt.imshow(
    amplitudes,
    vmin=0.0,
    vmax=1.0 * amplitudes.max(),
    extent=extents,
    origin="lower",
    cmap="nipy_spectral_r"
)
plt.xlabel("x (m)")
plt.ylabel("y (m)")
plt.title(f'Circular Diffraction (Monte Carlo);\nN={N},z={Z}')
plt.colorbar()
plt.show()

return

```

IV. RESULTS & DISCUSSION

We begin the analysis by testing the physics of the setup. Our main variable is observation distance Z . At small Z , this value should present the dynamic Fresnel pattern, and for larger Z , the stable Fraunhofer result. We can see in figure 1 a clear jaggedness and complex behaviour for the diffraction pattern. This is because our value of Z is small enough that we are deep within the Fresnel regime, allowing the complex term in Eq 1 to be dominant. When we increase Z to a much larger value, we see a much more typical Fraunhofer diffraction pattern, a smooth function that is our idealised fourier transform of our aperture shape ($\text{sinc}(x)$).

We next look to understand the role that the aperture size has on our patterns. The exercise script requests two specific sets of values, being when aperture size is 2×10^{-4} m with $Z = 0.005$ m (Fig 3), and an aperture size of 2×10^{-5} m with $Z = 0.05$ m (Fig 4). Figure 3 shows once again a very complex behaviour, characteristic of both a larger aperture and a smaller observation distance. Figure 4 shows our Fraunhofer behaviour, as Z is much larger and aperture size much smaller. We can quantify this using the Fresnel number

$$F = \frac{a^2}{\lambda Z} \quad (3)$$

where in our case, a is the half-width of the aperture (the characteristic size.) For $F \gg 1$ we are in the Near-Field regime, for $F \ll 1$ we are in the Far-Field regime, and for $F \approx 1$ we are in the transition period. For our first scenario, we calculate $F = 3.33$, firmly within the near-field behaviour, demonstrated by our complex jagged plot. The second scenario has a Fresnel number $F = 0.0033$, within the far-field behaviour, shown by the plot being effectively the characteristic Fraunhofer limit.

We also look to the workings of the `scipy.integrate.dblquad` function. There are two tolerance values that we can modify, `epsrel` and `epsabs`. Our actual tolerance is given $E \leq \text{epsabs} + \text{epsrel} \times |I|$. `epsabs` dominates when I is very close to zero, i.e. at the dark fringes, whereas `epsrel` dominates for high I , i.e. at the central maximum. Specifying these values to 1×10^{-10} allows us to reduce visual artifacts. Compare figures 5 and 6. No specification of the tolerances leads to visual artifacts or "spikes" at the edge of the plot, but tightening the tolerances removed any

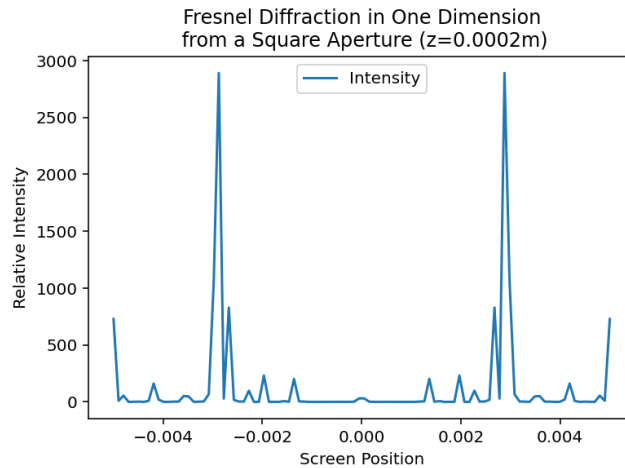


FIG. 1. The 1-d diffraction pattern for a rectangular aperture observed from a distance of $Z = 0.02$ m. This is an example of the near-field Fresnel pattern.

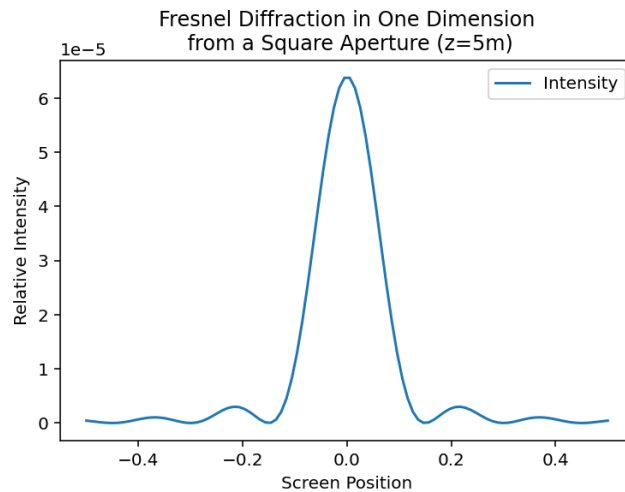


FIG. 2. The 1-d diffraction pattern for a rectangular aperture observed from a distance of $Z = 5$ m. This is an example of the far-field Fraunhofer pattern.

such artifacts, giving a much cleaner description of the physics, without any errors generated through computational limitations.

We can now turn our attention to 2-d plots. We look to generate a reference point displaying our Fraunhofer behaviour. We do this by running `part2()` with a large $Z = 0.1$ m. This displays our Fraunhofer pattern (Figure 7, characterised by a bright central maximum, separated "lobes" of light as you move from the centre, and a "cross" shape. This "cross" occurs due to the fourier transform of a rectangular aperture which yields intensity being defined as $\text{sinc}^2(x) + \text{sinc}^2(y)$. When x is large, $\text{sinc}(y)$ is only visible if $y \approx 0$ and vice-versa.

We can use our Fraunhofer behaviour to test our computational methods. Our primary test is comparing our direct Gaussian Quadrature method to our Monte Carlo approximation. Setting `numpoints` to 50 for both algorithms gives us a direct comparison. Figure 8 shows a clear central maximum with precise circular rings, whereas Figure 9 shows a lot more noise which is easily visible in the outer rings.

Noise is the main issue with the Monte Carlo method. The level of noise is determined by how many random samples N we take for each point. Compare Figures 9 and 10. We can see that for a small N , the pattern becomes almost unreadable, with the only consistency being the central maximum. Increasing N decreases the noise, and allows our approximation method to actually be useful. Increasing N by this amount does not have a significant increase in computation time, with $N = 100$ running for 0.26 s and $N = 10000$ running for just 1.45 s. This time

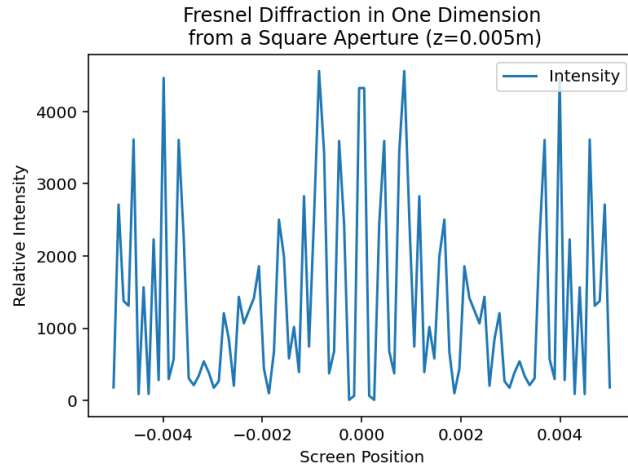


FIG. 3. 1-d diffraction pattern for a rectangular aperture of size 2×10^{-4} m observed from a distance of $Z = 0.005$ m. This is an example of the very complex near-field Fresnel pattern.

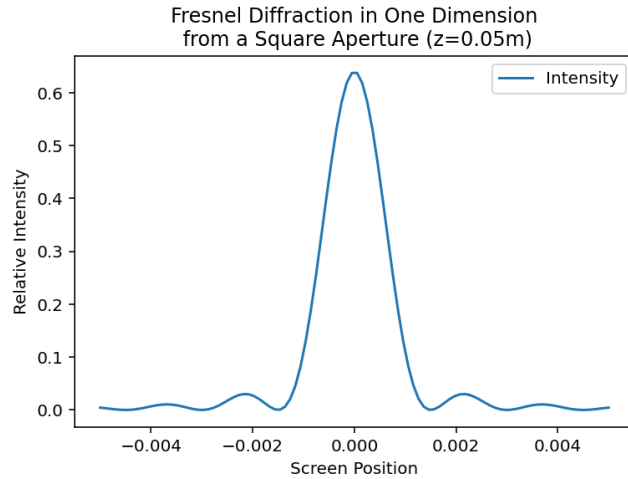


FIG. 4. The 1-d diffraction pattern for a rectangular aperture of size 2×10^{-4} m observed from a distance of $Z = 0.005$ m. This is an example of the near-field Fresnel pattern.

complexity is represented by $O(1/\sqrt{N})$, meaning that increasing N is computationally worth it for clear results.

We also look to compare runtimes between the two methods. A direct integration such as our Gaussian Quadrature method is expected to have a longer runtime. The main factor that increases runtime is increasing our `numpoints` value, as increasing the size of our aperture increases the number of pixels by the squared increase. We can compare Figures 11 and 12. These patterns are visually very similar, with the Monte Carlo pattern only having some slight noise artifacts, but on the whole the correct behaviour is observed in either figure. The difference between the usability of these methods is their runtimes: the Monte Carlo method had a reasonable runtime of 20.74 s whereas the Gaussian Quadrature method had an incredibly tedious runtime of 361.36 s. This shows that using our more accurate numerical integration method is often times unnecessary when a small noise trade-off can lead to accurate results with a much more manageable runtime.

V. CONCLUSION

The main takeaways from this exercise have been that the transition between near-field and far-field regimes is governed by the Fresnel Number (eq 3). We notice our Fraunhofer pattern for small F (usually with large Z), where

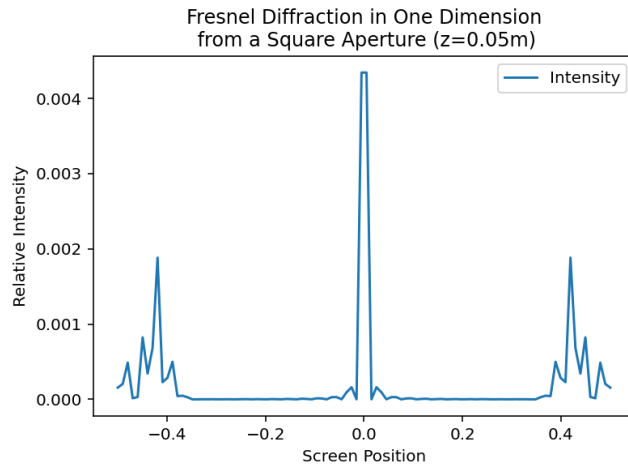


FIG. 5. Gaussian Quadrature pattern with aperture size $= 2 \times 10^{-5}$, $Z = 0.05$ m, and the default error parameters.

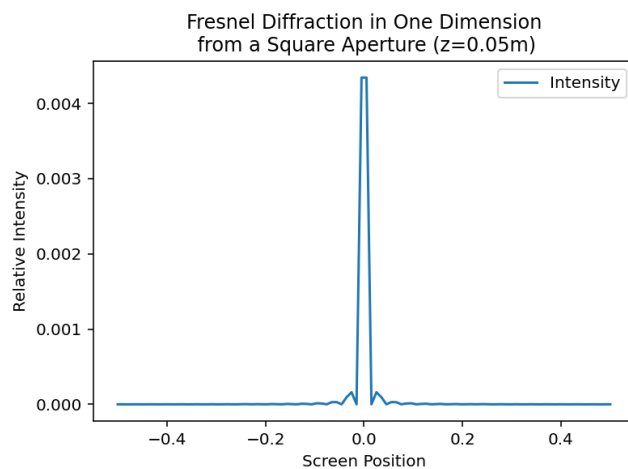


FIG. 6. Gaussian Quadrature pattern with aperture size $= 2 \times 10^{-5}$, $Z = 0.05$ m, and the error parameters set to 1×10^{-10} .

the pattern simplifies to the Fourier transform of the given aperture shape ($\text{sinc}^2 \times \text{sinc}^2$ or the disc shape). For larger F (usually with small Z) we noticed our complex and rapidly changing patterns. We also find a list of pros and cons for the different computational methods used. Gaussian Quadrature produces accurate and noise-less results in exchange for extreme runtimes that scale $O(N^2)$, whereas the Monte Carlo method is significantly faster for higher resolutions, but produces statistical noise that can be reduced by increasing the sample count, with the convergence of this scaling as $O(1/\sqrt{N})$. We also see that modifying tolerances in the `scipy.integrate.dblquad` function help to remove calculation artifacts produced where I is near zero. We also showed through the two scenarios given in the script that physical behaviour is not governed just by aperture size or observation distance, but by their ratio in the Fresnel number.

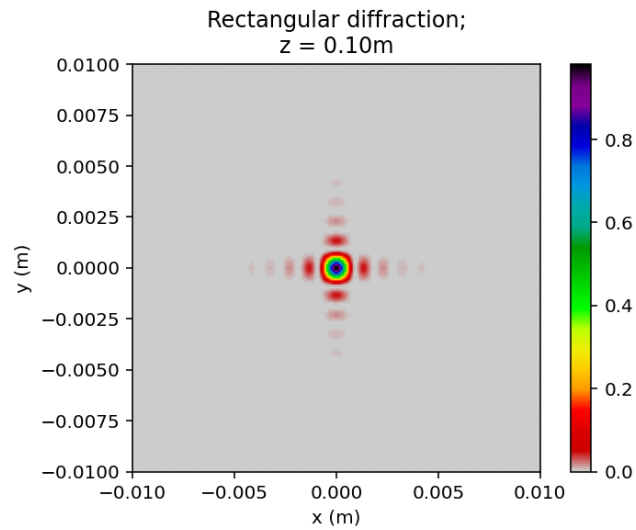


FIG. 7. The 2-d diffraction pattern of a rectangular aperture with a large Z . This is the characteristic Fraunhofer pattern.

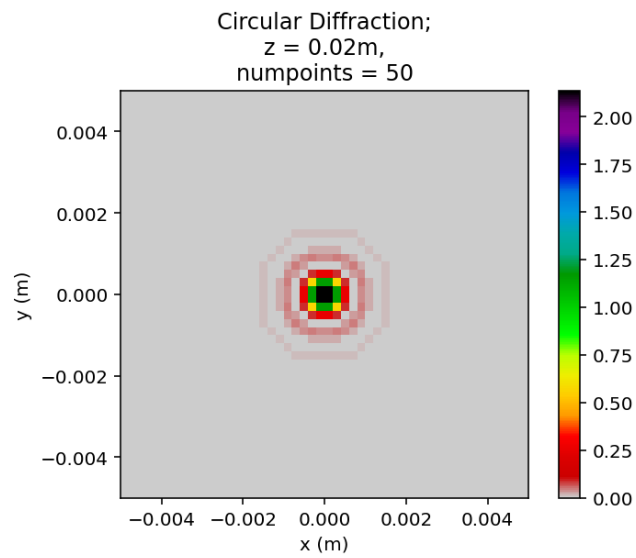


FIG. 8. The 2-d diffraction pattern of a rectangular aperture with a small Z and numpoints = 50 using the Gaussian Quadrature method.

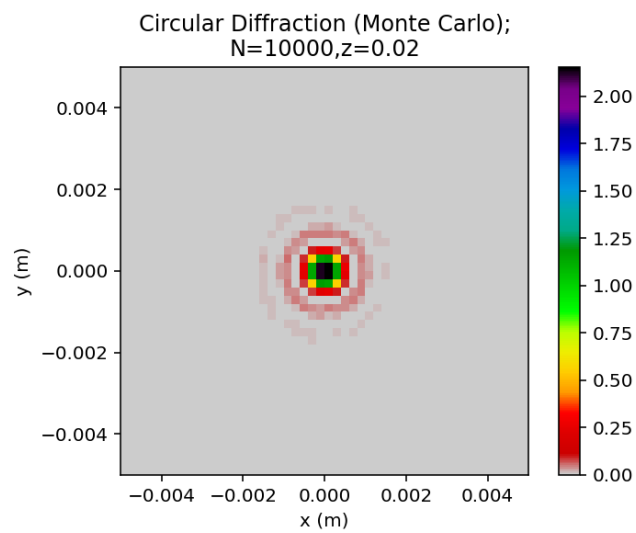


FIG. 9. The 2-d diffraction pattern of a rectangular aperture with a small Z and numpoints = 50 using the Monte Carlo method.

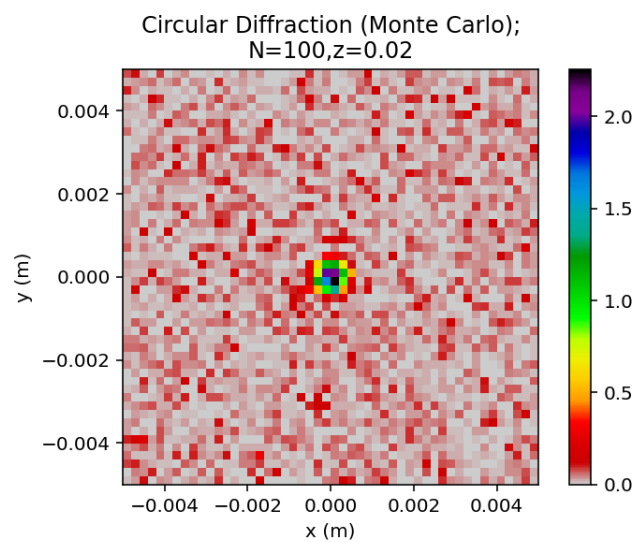


FIG. 10. Monte Carlo pattern with a small $N = 100$

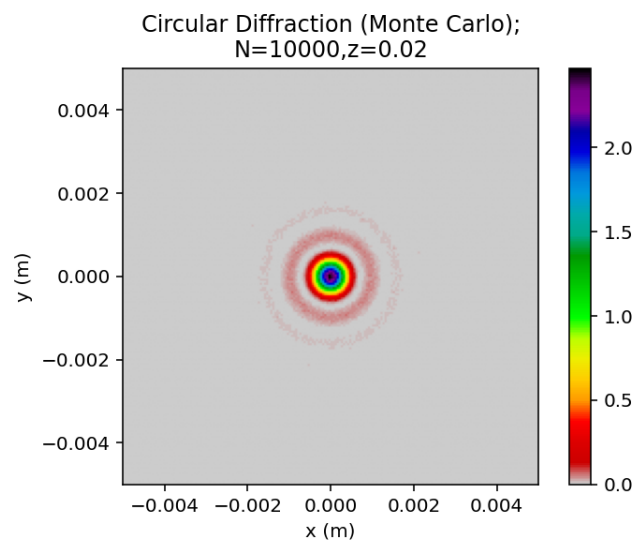


FIG. 11. Monte Carlo pattern with numpoints = 200

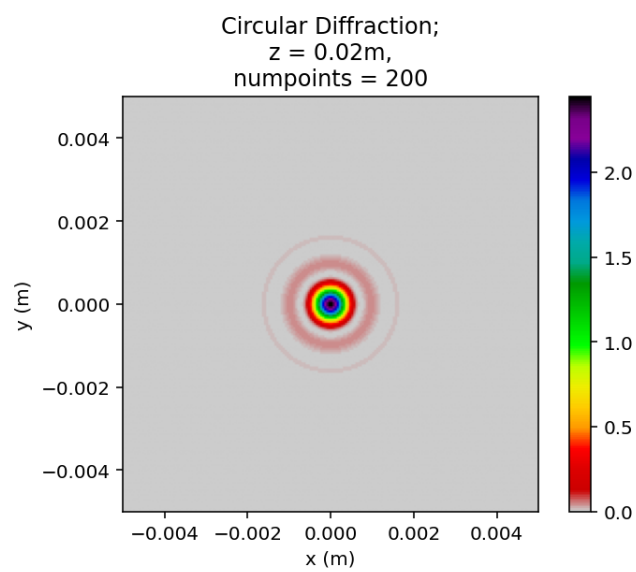


FIG. 12. Gaussian Quadrature pattern with numpoints = 200